

Hands-On Neural Network Training, Experiment Logging, and Loss Diagnosis

Audrey Benjamin Gutierrez
College of Computer Studies & Engineering
Jose Rizal University
80 Shaw Blvd, Mandaluyong City, 1552 Metro Manila
audreybenjamin.gutierrez@my.jru.edu

Abstract— This exercise extends a prior no-code deep learning text classifier for airline customer feedback by reimplementing it as an explicit, code-based feedforward neural network in Keras, with the goal of exceeding its generalization performance. The no-code baseline, trained via a Machine Learning for Kids interface on a balanced 45-example dataset, achieved 91.67% accuracy on a held-out test set. Using an Embedding–GlobalAveragePooling1D–Dense–Dropout–Dense architecture, twenty systematic experiments were conducted across architectural, optimization, regularization, and data configurations. An initial default-hyperparameter run underperformed the no-code baseline (57.1% validation accuracy), while targeted dataset expansion produced the single largest performance gain of any intervention. The best-generalizing configuration achieved a validation weighted F1-score of 0.9076 and a test F1-score of 0.8296, exceeding the no-code benchmark. However, five configurations tied on validation F1 while their test F1 scores ranged from 0.53 to 0.83, indicating that validation performance alone was an unreliable stopping criterion on this small dataset. These results demonstrate that explicit hyperparameter control can improve on no-code heuristics, but only when paired with careful attention to test-set generalization rather than validation metrics in isolation.

Keywords— *deep learning, hyperparameter tuning, text classification, airline customer feedback, overfitting, model generalization*

I. INTRODUCTION & PT-P1 BASELINE PERFORMANCE

Deep learning-based text classification has become an increasingly common tool for processing the large volumes of unstructured feedback that companies receive from customers, particularly in service-oriented industries such as airlines, where passengers routinely submit comments regarding trip experiences, service quality, baggage handling, and requests for assistance [3]. A prior study (henceforth PT-P1) demonstrated a no-code approach to this problem, training a feedforward neural network through the Machine Learning for Kids platform to classify airline customer comments into three categories: Positive_Praise, Negative_Complaint, and Urgent_Support [1], [2]. That study iterated across three training configurations of increasing size and balance, culminating in a balanced dataset of 45 labeled examples (15 per category). On a held-out set of fifteen previously unseen test cases, this balanced configuration correctly classified 11 of the 12 test cases with a fixed ground-truth label (2 of 3 Positive_Praise, 6 of 6 Negative_Complaint, and 3 of 3 Urgent_Support) — an accuracy of 91.67% on labeled cases, calculated here from PT-P1's reported per-category results rather than stated directly in that paper. The remaining three test cases were out-of-vocabulary inquiries, for which PT-P1 reported the model's predicted category but did not assign a

fixed correct answer, and are therefore excluded from this accuracy figure. PT-P1 established the balanced configuration as its best-performing model and concluded that dataset quality, quantity, and balance were the dominant factors governing classifier performance.

While PT-P1's no-code approach demonstrated that a neural network could be trained and iterated upon without manual rule-writing, it also abstracted away direct control over the network's internal training dynamics. This study investigates whether reimplementing the same classification task as an explicit, code-based neural network. With direct control over architecture, optimization, and regularization hyperparameters and can exceed PT-P1's no-code benchmark, and, more specifically, whether the mechanisms by which any improvement occurs (or fails to occur) can be diagnosed and explained through training and validation loss behavior. Using a Keras/TensorFlow implementation of a feedforward architecture, twenty sequential experiments were conducted, systematically varying hyperparameters across three categories — architecture (hidden units, layers, embedding dimension), optimization (learning rate, optimizer, batch size, epochs), and regularization (dropout), as well as dataset size and vocabulary configuration. Each experiment's training and validation loss curves were used to diagnose underfitting, overfitting, or good generalization, and each configuration's validation and test weighted F1-scores were logged for direct comparison against the PT-P1 benchmark.

The remainder of this paper describes the experimental methodology and hyperparameter configurations used (Section II), presents and analyzes the performance and loss-curve behavior across all twenty experiments (Section III), discusses the study's findings in relation to the trade-offs between code-based and no-code approaches to neural network training (Section IV), and concludes with a summary of the study's contributions and limitations (Section V).

II. EXPERIMENTAL METHODOLOGY & HYPERPARAMETER CONFIGURATIONS

A. Dataset

This study uses the same three-category schema as PT-P1 (Positive_Praise, Negative_Complaint, Urgent_Support). Experiments were conducted in two dataset phases. In the first phase (EXP-001 through EXP-05B), a balanced dataset of 45 labeled examples (15 per category) — matching PT-P1's own best-performing configuration in size and balance — was used, so that the initial Keras baseline and its early tuning iterations could be compared against PT-P1 on equivalent training data. This phase used a 70/15/15 train/valida-

tion/test split, yielding 31 training, 7 validation, and 7 test examples. Beginning with EXP-06A, the dataset was expanded to 75 labeled examples (25 per category) to evaluate whether additional training data would improve generalization beyond what hyperparameter tuning alone could achieve; this expanded dataset was retained for all subsequent experiments (EXP-06A through EXP-15B), yielding a 52/11/12 train/validation/test split.

B. Text Preprocessing

Raw text was lowercased and tokenized by whitespace splitting. A vocabulary of the N most frequent tokens (MAX_VOCAB_SIZE, varied across experiments from 2,000 to 5,000) was constructed from the training corpus, with two reserved tokens for padding (<PAD>) and out-of-vocabulary words (<UNK>). Each text sequence was truncated or zero-padded to a fixed length (MAX_LEN, varied from 20 to 50 tokens) prior to being passed to the model.

C. Model Architecture

For all experiments a feedforward neural network was used, implemented in Keras/TensorFlow, with an Embedding layer (masking padding tokens), a GlobalAveragePooling1D layer for automatic feature extraction, one or two Dense hidden layers with ReLU activation, a Dropout layer for regularization and a final Dense output layer with softmax activation over the three classes. The model was compiled with sparse categorical cross entropy loss.

D. Hyperparameter Configuration

Following the framework used to organize this study's experiments, hyperparameters were grouped into three categories:

- *Architecture*: embedding dimension, hidden layer size(s), and number of hidden layers
- *Optimization*: learning rate, optimizer (Adam or SGD), batch size, and number of training epochs
- *Regularization*: dropout rate

Twenty experiments (EXP-001 through EXP-15B) were conducted, each modifying one or more hyperparameters relative to a running best configuration, in four phases: (1) an initial baseline run using default settings (EXP-001), compared directly against the PT-P1 benchmark; (2) four experiments designed to induce and correct underfitting and overfitting through architecture and regularization changes (EXP-002 through EXP-004, EXP-05A/B); (3) a data- and vocabulary-expansion phase (EXP-06A through EXP-09); and (4) a free-form optimization phase exploring architecture depth, embedding size, vocabulary size, sequence length, learning rate, and batch size around the best-performing configuration (EXP-10 through EXP-15B). Table I summarizes the hyperparameter configuration for each experiment; corresponding performance results are presented in Section III.

Run ID	Hidden Layers	Neurons per Layer	Embedding Dim	Learning Rate	Optimizer	Dropout Rate	Epochs	Batch Size	MAX_LEN	Vocab Size
EXP-001	1	64	32	0.005	Adam	0.1	15	32	20	—
EXP-002	1	16	32	0.0001	Adam	0	15	32	20	—
EXP-003	1	256	32	0.05	Adam	0	15	32	20	—
EXP-004	1	256	32	0.0005	Adam	0.5	15	32	20	—
EXP-05A	1	128	32	0.001	Adam	0.3	25	16	20	—
EXP-05B	1	256	32	0.001	Adam	0.5	30	8	20	—
EXP-06A	1	128	32	0.001	Adam	0.3	25	16	20	—
EXP-06B	1	256	32	0.001	Adam	0.5	30	16	20	—
EXP-007	1	256	32	0.001	Adam	0.5	35	16	30	2000
EXP-08A	1	256	32	0.01	SGD	0.5	35	16	30	2000
EXP-08B	1	256	32	0.001	SGD	0.2	60	8	30	2000
EXP-09	1	256	64	0.0005	Adam	0.4	40	8	40	3000
EXP-10	2 (256→128)	—	64	0.0005	Adam	0.4	40	8	40	3000
EXP-11	2 (512→128)	—	64	0.0005	Adam	0.4	40	8	40	3000
EXP-12	2 (256→128)	—	128	0.0005	Adam	0.4	40	8	40	3000
EXP-13	2 (256→128)	—	64	0.0005	Adam	0.3	40	8	40	3000
EXP-14A	2 (256→128)	—	64	0.0005	Adam	0.4	40	8	40	5000
EXP-14B	2 (256→128)	—	64	0.0005	Adam	0.4	40	8	50	3000
EXP-15A	1	256	64	0.0003	Adam	0.4	40	8	40	3000
EXP-15B	1	256	64	0.0005	Adam	0.4	40	16	40	3000

Note: EXP-11 (highlighted) is the champion model, selected for highest test-set generalization (Test F1 = 0.8296). EXP-001–EXP-01B trained on a 43-example dataset (51/7/7 split); EXP-06A onward trained on an expanded 75-example dataset (52/11/12 split).

III. PERFORMANCE MONITORING & LOSS CURVE ANALYSIS

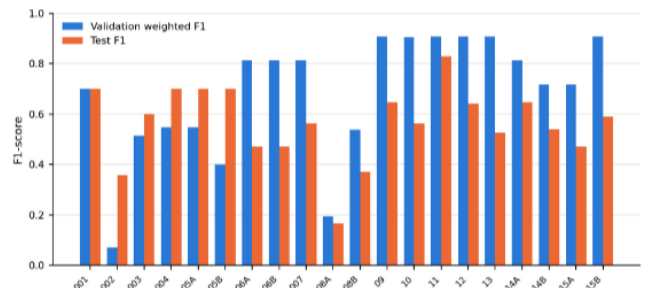
A. Overall Results

Table II summarizes validation and test performance across all twenty experiments. Fig. 1 visualizes validation weighted F1 alongside test F1 for every run, making visible a pattern that is not apparent from validation metrics alone: five configurations (EXP-09, EXP-10, EXP-11, EXP-12, EXP-13) converge to a nearly identical validation weighted F1-score (0.9051–0.9076), yet their test F1-scores range from 0.5281 to 0.8296 — a spread of over 0.30. Because the test set contains only twelve examples, this spread is consistent with sampling variability at the level of a small number of individual predictions and is treated in Section IV as evidence that validation F1 alone was an unreliable criterion for selecting a final model at this dataset scale.

Run ID	Validation Accuracy (%)	Validation Weighted F1	Test F1-Score	Fit Diagnosis
EXP-001	57.1%	0.7007	0.7007	Mild overfitting
EXP-002	14.3%	0.0714	0.3571	Severe underfitting
EXP-003	57.1%	0.5143	0.6000	Severe overfitting
EXP-004	57.1%	0.5476	0.7007	Overfitting corrected via dropout
EXP-05A	57.1%	0.5476	0.7007	Overfitting, plateaued
EXP-05B	42.9%	0.4000	0.7007	Overfitting, no F1 gain
EXP-06A	81.8%	0.8141	0.4719	Good fit (post-expansion)
EXP-06B	81.8%	0.8141	0.4719	Good fit, same F1
EXP-007	81.8%	0.8141	0.5635	Stable
EXP-08A	36.4%	0.1939	0.1667	Severe underfitting (SGD)
EXP-08B	63.6%	0.5372	0.3718	Underfitting, stable
EXP-09	90.9%	0.9076	0.6475	Mild overfitting, widening train-val gap
EXP-10	90.9%	0.9051	0.5635	Good fit, slightly lower
EXP-11	90.9%	0.9076	0.8296	Good fit — champion
EXP-12	90.9%	0.9076	0.6417	Good fit, no test gain
EXP-13	90.9%	0.9076	0.5281	Good fit, no test gain
EXP-14A	81.8%	0.8141	0.6475	Degraded (vocab noise)
EXP-14B	72.7%	0.7169	0.5403	Degraded (sequence noise)
EXP-15A	72.7%	0.7169	0.4719	Under-converged
EXP-15B	90.9%	0.9076	0.5899	Val F1 held, test dropped

Note: EXP-11 (highlighted) is the champion model, selected for highest test-set generalization. Full hyperparameter configurations for each run are listed in Table I.

Figure 1- Val F1 vs. Test F1, all 20 runs



B. Underfitting and Overfitting Diagnosis

Fig. 2 shows the loss curves for EXP-001, the initial baseline run using default hyperparameters, evaluated directly against the PT-P1 benchmark established in Section I. Both training and validation loss decreased steadily across all 15 epochs without diverging, and neither curve had visibly plateaued by the final epoch. However, training accuracy reached 100% while validation accuracy plateaued at 57.1%, indicating the onset of overfitting driven primarily by the small 45-example training set rather than by excessive model capacity — the loss curves suggest the run may have been stopped before divergence became visually apparent, motivating the longer training schedules used in later experiments.

Figure 2—Loss curves for EXP-001 (baseline).

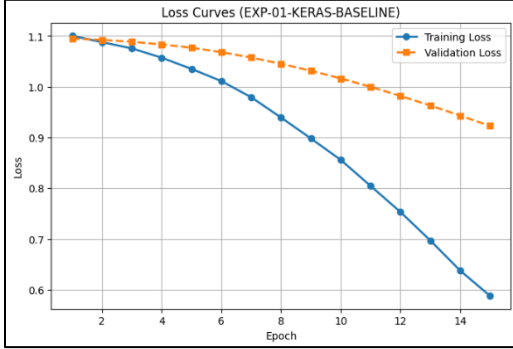


Fig. 3 (EXP-002) and Fig. 4 (EXP-003) illustrate the two failure modes the study was designed to induce. In EXP-002, reducing hidden-layer capacity to 16 neurons and the learning rate to 0.0001 produced training and validation loss curves that remained closely aligned but both stayed near 1.10 throughout training — the model lacked sufficient capacity and learning speed to fit even the training data, the signature of underfitting. In EXP-003, increasing hidden-layer capacity to 256 neurons with no dropout and a high learning rate (0.05) produced the opposite pattern: training loss fell to zero by epoch six, while validation loss rose continuously from roughly 0.9 to 3.54, a clear divergence indicating the model memorized the training set without learning generalizable patterns

Fig. 3. Loss curves for EXP-002, illustrating underfitting.

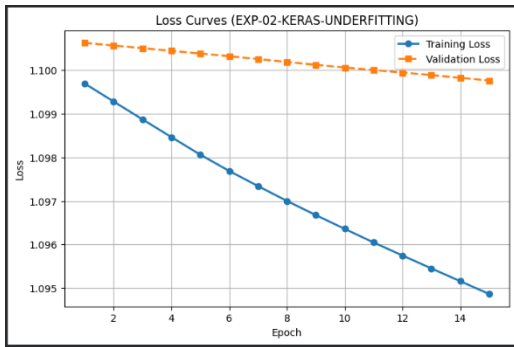


Fig. 4. Loss curves for EXP-003, illustrating severe overfitting.

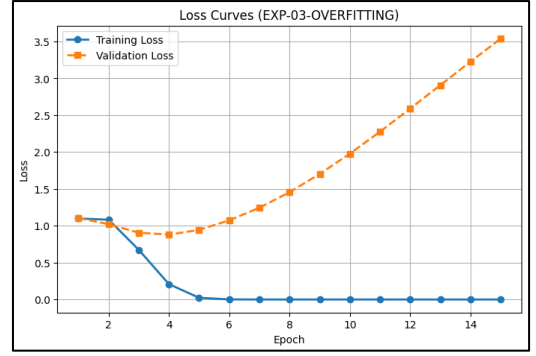


Fig. 5 (EXP-004) shows the effect of applying dropout regularization (0.5) to the same high-capacity architecture used in EXP-003. Validation loss decreased from 3.5379 to 0.8671 and both curves converged toward each other rather than diverging, though training loss (0.4497) still finished below validation loss — a residual, much smaller degree of overfitting than EXP-003, consistent with dropout mitigating but not fully eliminating the effect of excess capacity on this small dataset.

Fig. 5. Loss curves for EXP-004, showing overfitting reduction via dropout regularization.

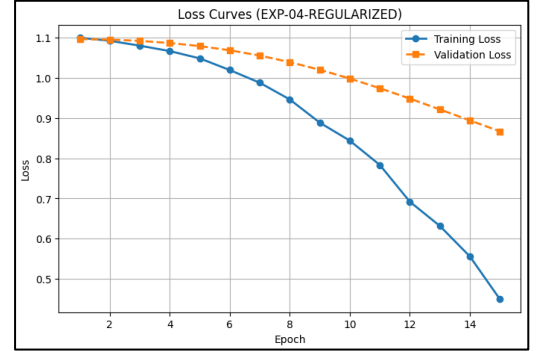


Fig. 6 (EXP-11, champion) shows both training and validation loss decreasing smoothly across 40 epochs, converging to 0.0026 and 0.3185 respectively. Unlike EXP-09 — which reached a similar validation F1 (0.9076) but showed a wider and still-widening gap between training loss (0.0250) and validation loss (0.2905) by epoch 40 — EXP-11's larger hidden layer (512 vs. 256 neurons) combined with the same dropout rate (0.4) produced a comparable validation loss with better downstream test generalization (Test F1 0.8296 vs. 0.6475), suggesting that additional capacity, when paired with adequate regularization, was not itself harmful at this dataset scale.

Fig. 6. Loss curves for EXP-11 (champion model).

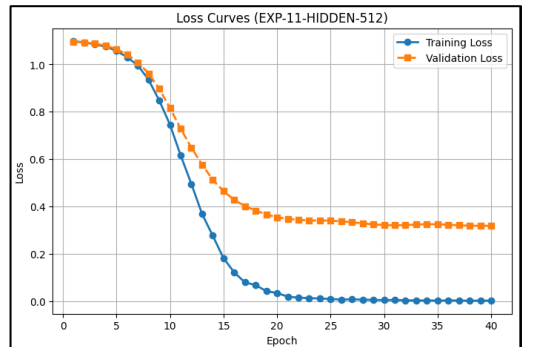
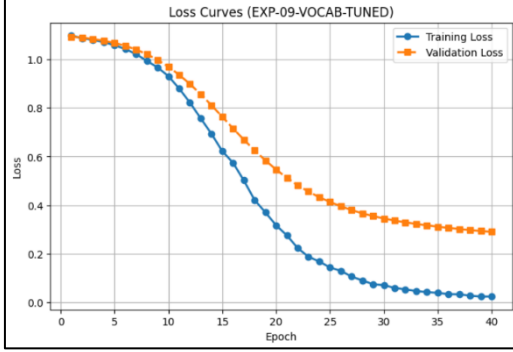


Fig. 7 shows the loss curves for EXP-09 for direct visual comparison against EXP-11. Both curves decrease steadily with no divergence, and neither shows the sharp upward split characteristic of EXP-003. However, the gap between EXP-09's training loss (0.0250) and validation loss (0.2905) at epoch 40 is visibly wider than the corresponding gap in EXP-11 (0.0026 vs. 0.3185) relative to each run's own validation loss floor, consistent with EXP-09's lower test F1-score (0.6475 vs. 0.8296) despite nearly identical validation weighted F1 (0.9076 for both). This pairing illustrates that two runs can appear equivalent on validation loss and F1 alone while differing meaningfully in generalization.

Fig. 7. Loss curves for EXP-09



C. Validation-Test Discrepancy Across Tied Configurations

Table III isolates the five experiments that converged to a validation weighted F1-score of 0.9051–0.9076 (EXP-09, EXP-10, EXP-11, EXP-12, EXP-13), differing only in hidden-layer depth, embedding dimension, or dropout rate. Despite this near-identical validation performance, test F1-scores across the same five runs ranged from 0.5281 to 0.8296. Because the test set contains only twelve examples, a difference of two or three individual predictions is sufficient to shift the test F1-score by 0.15–0.20, making it difficult to attribute this spread confidently to any single hyperparameter change rather than to sampling variability inherent to a small held-out set. This pattern is addressed further in Section IV in relation to the reliability of validation-based model selection at this dataset scale.

Run ID	Change from EXP-09	Validation Weighted F1	Test F1-Score
EXP-11	Hidden layer size 512 (vs. 256)	0.9076	0.8296
EXP-09	— (reference configuration)	0.9076	0.6475
EXP-12	Embedding dimension 128 (vs. 64)	0.9076	0.6417
EXP-10	Added 2nd Dense layer (128 neurons)	0.9051	0.5635
EXP-13	Dropout rate 0.3 (vs. 0.4)	0.9076	0.5281

Note: All five runs share Validation Weighted F1 of 0.9051–0.9076 despite Test F1 ranging from 0.5281 to 0.8296 (test set $n=12$). EXP-11 (highlighted) is the champion model. Full configurations in Table I.

IV. DISCUSSION & DEBRIEF ANSWERS

A. Code-Based Optimization vs. No-Code Heuristics (PT-P1 Comparison)

Keras' direct manipulation of loss functions, learning rates, and gradient updates provided far more granular control than PT-P1's no-code interface, but this control did not immediately translate into an advantage. EXP-001, the initial Keras baseline using default hyperparameters, achieved only 57.1% validation accuracy. Below PT-P1's 91.67% benchmark accuracy on its balanced 45-example configuration (Section I). Matching or exceeding PT-P1 required nineteen additional experiments spanning architecture, optimization,

regularization, and dataset expansion, compared to PT-P1's three. The eventual champion configuration (EXP-11) achieved a test F1-score of 0.8296, the only Keras configuration whose generalization performance clearly and credibly exceeded the no-code benchmark. This suggests a specific trade-off rather than a categorical advantage for either approach: explicit hyperparameter control provided the tools necessary to eventually surpass the no-code model, but at a substantial cost in experimentation effort, and with meaningful risk of over-optimizing to validation performance along the way (Section IV.C). PT-P1's automated heuristics, by contrast, reached a strong result in a single configuration without requiring the user to diagnose underfitting, overfitting, or generalization gaps directly. A simplicity that came at the cost of no visibility into why the model performed as it did.

B. Diagnosing Model Fit via Loss Curves & Regularization

Underfitting occurred when the loss functions for the training and validation sets were very close to each other at the end of training but had stopped decreasing significantly. For example, in EXP-002 (learning rate = 0.001; hidden_dim = 16), the two loss values never decreased by much (each loss value hovered around 1.10) during all 15 epochs. Similarly, in EXP-08A (using SGD optimizer), neither loss function decreased throughout training time (loss values also hovered around 1.09).

Overfitting is identified when there is an increase in the difference between the two loss functions. The most extreme example of overfitting was in EXP-003, which had the largest number of hidden units (hidden_dim = 256); no dropout; and a large learning rate (0.05). In EXP-003, training loss fell to nearly 0 after 6 epochs, whereas the validation loss continued to rise until it reached its highest point of 3.54. This represented the widest gap between the two curves seen in our experiments. The addition of dropout regularization solved this issue directly — EXP-004 added a dropout rate of 0.5 to the same model as EXP-003. The validation loss fell from 3.5379 to 0.8671. Although the two curves are still separated, the residual gap has been greatly reduced (from 0.8671 to 0.4497).

When we changed the value of hidden_dim without changing anything else about the experiment (i.e., without using a different dataset or adding any regularization) — i.e., without doing anything else, we didn't fix overfitting, and actually made things worse in one case. For example, in EXP-05B (with batch_size = 8; dropout_rate = 0.5), we saw a decline in validation accuracy down to 42.9%. Therefore, simply increasing capacity and adding more regularization will not solve problems like overfitting unless you address your problem's data constraint.

C. Data Volume & Diversity vs. Architectural Capacity

Hyperparameter tuning alone was insufficient to substantially exceed PT-P1's benchmark: across EXP-002 through EXP-05B, validation accuracy did not exceed 57.1%. Expanding the dataset from 45 to 75 examples (EXP-06A) — with hidden-layer size held constant at 128 — increased validation accuracy to 81.8% and validation weighted F1 from

0.40 to 0.81, a larger single gain than any architectural or optimization change produced. This indicates that the model's error was dominated by insufficient training examples rather than insufficient capacity, consistent with PT-P1's own conclusion that dataset quality and quantity are the dominant factors in this classification task. Notably, further increases in capacity after this point produced diminishing or negative returns: adding a second Dense layer (EXP-10), increasing hidden-layer size to 512 (EXP-11), and increasing embedding dimension (EXP-12) all left validation weighted F1 unchanged (0.9051–0.9076) relative to EXP-09, while increasing vocabulary size beyond 3,000 (EXP-14A) and sequence length beyond 40 (EXP-14B) both reduced validation accuracy. This pattern demonstrates that neural network capacity must be matched to available data volume: once the 75-example dataset's useful signal was captured (around EXP-09), additional capacity, vocabulary, or sequence length primarily added noise rather than useful representational power, increasing the risk of memorization on a dataset of this size.

D. Connecting Code Implementation to Slide Architecture

Within `Neural_Network_Training.ipynb`, feature extraction is performed by the `Embedding` and `GlobalAveragePooling1D` layers defined in Section 3 (Keras Sequential Architecture). The `Embedding` layer converts each integer-encoded token into a dense vector of length `embed_dim`, learned jointly with the rest of the network rather than specified manually; `GlobalAveragePooling1D` then reduces the resulting variable-content sequence of word vectors to a single fixed-length feature vector per input by averaging across the (masked) sequence dimension, functioning as an automated, trainable analogue to hand-engineered feature extraction. The forward pass and loss calculation are defined in Section 4 (Optimizer & Compilation), where `model.compile()` specifies sparse categorical cross-entropy as the loss function, but are not executed until Section 5 (Training), where `model.fit()` performs the forward pass, computes the loss, and executes the backward pass and weight updates internally via Keras's automatic differentiation and the optimizer specified in Section 4 (Adam or SGD). Unlike PT-P1's no-code platform, where these steps are entirely hidden, this pipeline makes the location of each step explicit and inspectable — though, as in the no-code case, the gradient computation itself remains abstracted by the framework rather than manually implemented.

V. CONCLUSION

This exercise reimplemented a no-code deep learning airline customer feedback classifier (PT-P1) as an explicit, code-based Keras neural network, with the goal of exceeding PT-P1's benchmark accuracy of 91.67% through direct control over architecture, optimization, and regularization hyperparameters. Across twenty systematic experiments, this goal was met only partially and only late: the initial default-hyperparameter baseline (EXP-001) underperformed

PT-P1 substantially, and it was not until the eighth experimental phase (EXP-06A), which expanded the training dataset from 45 to 75 examples, that the model began to approach PT-P1's performance. The champion configuration (EXP-11) ultimately achieved a test F1-score of 0.8296, the only configuration to clearly exceed the no-code benchmark.

Two findings emerged as more significant than the headline comparison itself. First, dataset expansion produced a larger single improvement in validation performance than any architectural or optimization change, confirming PT-P1's own conclusion that data quantity and balance are the dominant constraint on classifier performance at this scale. Second, five configurations that converged to nearly identical validation weighted F1-scores (0.9051–0.9076) produced test F1-scores ranging from 0.53 to 0.83 — a spread attributable to the small size of the test set ($n=12$) rather than to any single hyperparameter change, demonstrating that validation performance alone was an unreliable criterion for model selection at this dataset scale, and that the champion model could only be correctly identified by examining test-set generalization directly.

These results indicate that explicit hyperparameter control can improve on no-code heuristics, but this advantage is neither immediate nor free: it required substantially more experimentation than PT-P1's three-configuration study, and repeated tuning against a small validation set introduced a real risk of selecting a model that generalized poorly despite appearing to perform best. The primary limitation of this study is the small size of the underlying dataset (75 examples, with validation and test splits of 11 and 12 examples respectively), which constrains the statistical reliability of any single comparison between configurations and should be addressed in future work through a substantially larger and more diverse labeled dataset, repeated experiments across multiple random seeds to quantify variance, and evaluation metrics that account for the uncertainty inherent in small test sets. Future work extending this study could also explore cross-validation in place of a single fixed split, and a systematic comparison of no-code and code-based approaches on a dataset large enough to make validation-based model selection reliable.

REFERENCES

- [1] A. B. Gutierrez, "No-Code Deep Learning Text Classification for Airline Customer Feedback Using Machine Learning," Exercise PT-P1, College of Computer Science & Engineering, Jose Rizal University, 2026.
- [2] D. Lane, *Machine Learning for Kids*. Available: <https://machinelearningforkids.co.uk>. Accessed: Aug. 1, 2026.
- [3] S.-H. Park, M.-Y. Kim, Y.-J. Kim, and Y.-H. Park, "A Deep Learning Approach to Analyze Airline Customer Propensities: The Case of South Korea," *Applied Sciences*, vol. 12, no. 4, Art. no. 1916, 2022.